

```
22 WRITE_REG(hw, RDT, E1000_NUM_RX_DESC - 1);
23 }
```

代码片段 13.10 将配置 DMA Rx 的逻辑加入了设备驱动的初始化流程，同时在 Rx 中断发生时从 DMA 描述符中获取数据帧的信息。在 Rx 中断发生时，Rx 中断处理函数会遍历 Rx 队列，找出所有就绪的数据帧，从描述符中获取对应的地址和长度，从而得到数据帧并传给操作系统的网络协议栈进行处理。

代码片段 13.10 基于 DMA 队列的 e1000 的中断处理

```
1 #define E1000_RXD_STAT_DD 0x01      // 描述符就绪
2 #define E1000_ICR_RXT0 0x80         // Rx 中断
3
4 int e1000_irq_handler(struct e1000_hw *hw)
5 {
6     ...
7     // 判断是否为 e1000 Rx 中断
8     unsigned irq = READ_REG(hw, E1000_ICR);
9     if (irq & E1000_ICR_RXT0) {
10         while (1) {
11             // 读取 Rx 尾指针，尾指针的下一个为可用的 Rx 数据帧
12             int rx_cur = READ_REG(hw, RDT);
13             rx_cur = (rx_cur + 1) % RXBUF_COUNT;
14
15             // 判断当前 Rx 缓冲区的数据是否就绪
16             if (!(rx_base[rx_cur]->status & E1000_RXD_STAT_DD))
17                 return ERROR;
18
19             // 将网络包交给操作系统的网络协议栈进行处理
20             uint64_t buf = rx_base[rx_cur]->addr;
21             uint16_t len = rx_base[rx_cur]->length;
22
23             net_rx((char *)buf, len);
24             rx_base[rx_cur]->status = 0;
25
26             WRITE_REG(hw, RDT, rx_cur);    // 更新 Rx 尾指针
27         }
28         return SUCCESS;
29     }
30     ...
31 }
32
33 void e1000_init(struct e1000_hw *hw)
34 {
35     int irq;
36     register_interrupt(&irq, e1000_irq_handler);
37     ...
38     e1000_setup_rx_ring(hw);
39 }
```

13.3 设备管理的共性功能

本节主要知识点

- 应用程序是如何找到设备的？
- 为什么要对设备进行分类？不同分类有什么特点？
- 为什么要设计不同的缓冲区？缓冲区是否是必要的？
- 为什么要提出 `ioctl` 系统调用？

尽管不同设备的功能各异，但从管理的角度来看，它们之间有着不少共同的需求。首先，操作系统需要为所有设备提供合适的抽象，以便应用程序对设备进行查找和访问。其次，操作系统需要根据设备功能对设备进行合理的分类，以便提取共性需求并复用功能相同的实现代码。再次，操作系统需要根据设备的特点提供合适的管理方案，比如根据设备的访问特性、交互速率选择合适的缓冲区方案。最后，考虑到设备的多样性，操作系统需要为应用程序提供合适的设备操作接口。

13.3.1 设备的文件抽象

为了帮助应用程序和设备进行交互（比方应用程序希望通过串口输出计算结果），操作系统需要为设备设计合理的软件抽象，同时分配合适的逻辑名称，以方便应用程序查找相应设备。既然操作系统的用户已经对文件这一抽象十分熟悉，那么能不能直接复用这个抽象，用管理文件的方式管理设备呢？答案是肯定的。以 UNIX 操作系统为例，它的核心设计思想之一是一切都是文件，设备也不例外。为了对各种设备进行统一管理，操作系统将设备按照各自的特点进行分类，组织为树状的层次结构挂载到设备文件系统（Device Filesystem, DevFS）下（默认为 `/dev` 目录）。设备文件系统的叶子节点称为设备文件（Device File），对应设备实例，其完整的路径名则作为设备的逻辑名称。设备文件是操作系统对物理设备抽象而成的虚拟设备，例如在 Linux 系统上，显卡的显存被抽象成帧缓存（Framebuffer，路径名为 `/dev/fb`），串口设备被抽象成终端（Teletype，路径名为 `/dev/tty`）。

如图 13.17 所示，有了设备文件系统，应用程序就可以直接使用文件系统的 `open` 接口打开设备文件，使用 `read/write` 接口读写设备文件的内容。通过复用文件的方式提供 I/O 设备的操作接口至少有三个优点：

- 操作系统的用户已经习惯了文件的使用方法，因此采用文件系统接口处理 I/O 更符合直觉。
- 设备名遵循文件名的用法和语义，设备文件系统由此可以向应用程序提供统一的设备命名空间和索引方式。
- 设备文件复用文件系统的权限管理，通过访问控制机制决定进程可以访问哪些设备。

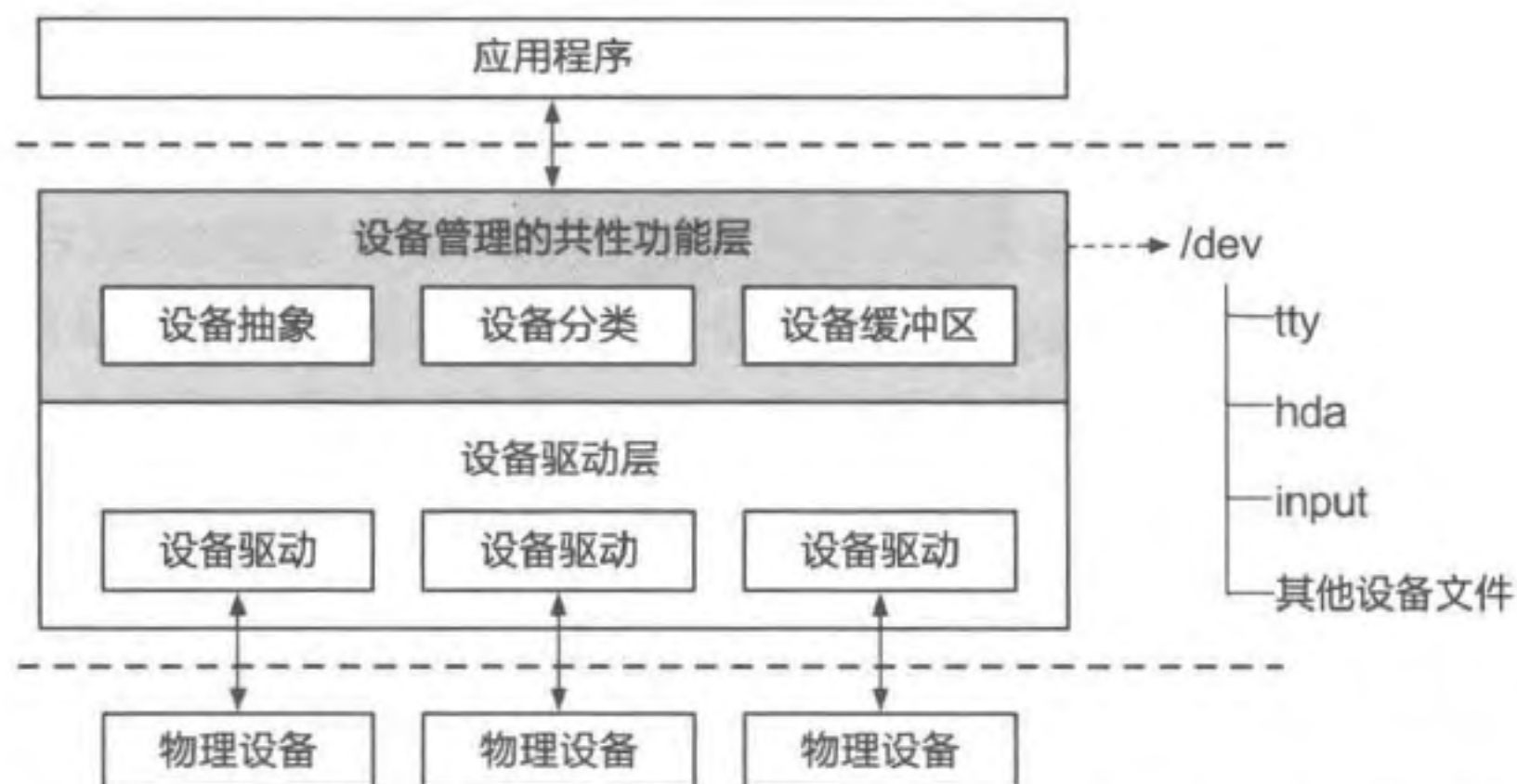


图 13.17 设备管理的共性功能层作为应用程序与设备驱动的连接层，对上提供设备文件系统抽象，对下将文件操作转化为驱动命令

为了将设备文件和普通文件的操作统一起来，操作系统需要在 inode 数据结构中加入和设备相关的标志。如代码片段 13.11 所示，设备文件所使用的 inode 是一类特殊的 inode，本身不指向任何真实的物理文件。

代码片段 13.11 设备文件和普通文件使用 inode 进行索引，通过文件类型调用对应的文件接口函数（write 操作的流程与 read 类似）

```

1 struct inode {
2     short type;           // 节点类型，用于区分普通文件和设备文件
3     short major;         // 主设备号，用于区分设备驱动，只对设备文件有效
4     short minor;         // 次设备号，用于区分设备实例，只对设备文件有效
5     ...
6 };
7
8 struct device_table { // 设备文件相关的读写操作
9     int (*read)(struct inode*, char *, int, int);
10    int (*write)(struct inode*, char *, int, int);
11 } device_table[];
12
13 void register_device(struct device *device, int device_type)
14 {
15     device_table[idev].write = device.ops.write;
16     device_table[idev].read = device.ops.read;
17     ...
18 }
19
20 int readi(struct inode *ip, char *dst, int off, int n)
21 {
22     if (ip->type == DEVICE_TYPE) {
23         if(NULL == device_table[ip->dev.major].read)
24             return -1;
25         return device_table[ip->dev.major].read(ip, dst, off, n);

```

```
26 }  
27 ...  
28 }
```

同一设备可能存在多个实例，例如主机上配置有多块存储设备，或者同时有多个USB接口。为了将设备文件锚定到这些具体的设备实例上，操作系统首先分配一个主设备号（Major Number），用于确定系统中对应的驱动程序，因为同一驱动程序可以操作相同类型设备的不同实例。然后分配一个次设备号（Minor Number），用于区分具体的设备实例。例如，串口设备 `/dev/tty` 和模拟串口的屏幕终端 `/dev/console` 都可以使用串口驱动进行操作，因此拥有相同的主设备号，但次设备号不同。

设备驱动在向操作系统注册自己的同时，还需要将设备自身的 `read/write` 回调函数注册到全局的 `device_table` 中。当应用程序打开设备文件并通过文件系统的 `read/write` 接口进行 I/O 操作时，操作系统会调用主设备号对应的驱动程序的 `read/write` 回调函数对设备进行操作。

13.3.2 设备的逻辑分类

由于设备的多样性，操作系统会根据设备的特点对设备进行分类，力图以相同的方式管理特点相似的设备。在不同的操作系统中，设备的分类方式也有所不同。例如，Linux 将设备大致分为三类：字符设备、块设备和网络设备。

- 字符设备（Character Device）。字符设备将设备的数据抽象为字节流，由应用程序以顺序访问的方式进行读写。字符设备使用标准文件系统接口，包括 `open`、`read`、`write`、`close` 等。串口、时钟、键盘、鼠标、声卡都被归类为字符设备。
- 块设备（Block Device）。块设备将设备上的数据划分为独立编址的块，由应用程序采用随机访问的方式对块进行读写。块是设备寻址的最小单元（通常为 512 字节、1KB 或 4KB）。块设备常见于对存储设备的抽象，例如虚拟磁盘（Ramdisk）、存储卡（SD 卡和 MMC 卡）、硬盘（HDD 和 SSD）等。

虽然块设备和字符设备一样可以使用标准的文件系统接口进行访问，但是应用程序往往更倾向于使用 `mmap` 接口，将块设备的文件映射到用户空间的内存中进行操作。为了避免频繁操作块设备引起的性能问题，操作系统通常会为块设备增加一层缓冲区。关于设备缓冲区的用法，我们将在下一节进行介绍。

- 网络设备（Network Device）。网络设备是一种以格式化报文（也称为分组或数据包）的形式进行数据通信的设备。这类设备使用一种特殊的抽象——套接字（Socket）。用户程序使用 `send`、`recv`、`sendto`、`recvfrom` 等专用网络设备接口，通过套接字进行数据传输。网络设备需要有特定的网络协议栈的支持才能处理网络报文的封装、解析、寻址等功能。常见的网络协议有以太网（Ethernet）、

无线网 (Wireless)、蓝牙 (Bluetooth) 等。

为了兼容文件抽象, 应用程序也可以使用文件系统接口 (如 `read/write`) 实现网络报文的收发。不同的是, 操作系统一般不会在设备文件系统目录 (`/dev`) 下为网络设备建立对应的网络文件, 因此应用程序通常无法通过 `open` 接口打开网络设备, 而必须通过 `socket` 接口创建套接字。

随着新设备的出现, 并非所有的新设备都能被很好地归类到这三类设备中。比如用于机器学习训练的通用 GPU 设备, 在 Linux 中属于独立设备, 同样以设备文件的形式挂载在 `/dev` 目录下, 但通常不使用 `read/write` 接口进行交互, 而是使用后文将要介绍的 `ioctl` 接口。

不同于 Linux, 同属于类 UNIX 操作系统的 FreeBSD 则弃用了块设备的抽象^[7]。FreeBSD 开发人员给出的理由是块设备提供了缓冲区机制, 使得应用程序难以追踪 I/O 问题, 进而容易导致系统和程序运行变得不可靠。因此, FreeBSD 不提供块设备抽象, 而是将硬盘等设备当成裸设备 (Raw Device) 直接暴露给应用程序, 并将裸设备和字符设备统称为字符设备。

13.3.3 设备的缓冲区管理

在对设备进行合理的逻辑分类之后, 操作系统还需要根据不同分类的设备特性保证和设备交互时的性能和正确性。请读者思考如下问题:

- 基于云存储的共享文档场景下, 多个用户想要同时修改存储设备上的同一个文件。操作系统该如何设计, 才能尽快将存储设备上的文件修改好?
- 运行手机游戏时, CPU 需要源源不断地将游戏进程产生的画面同步到显存中; 然而在显卡渲染时, CPU 不能对显存进行操作, 否则会产生画面错位。操作系统该如何设计, 才能实现较好的画面质量?
- 操作系统和网卡交互时, 需要在单位时间内尽可能多地传输网络帧; 操作系统通常需要等待网卡确认数据发出后才能覆写原先数据所占的内存空间。操作系统该如何设计, 才能实现更为高效的数据交互呢?

根据真实场景的具体需要, 操作系统可以实现不同类型的缓冲区, 比如单缓冲区、双缓冲区、多缓冲区等。

单缓冲区

很多设备的工作频率远远达不到 CPU 的主频, 如果每次操作设备都需要向真实设备发起操作, 那么整个系统的工作效率将会被这些慢速设备所拖累。因此, 操作系统在慢速设备和高速 CPU 之间引入一段内存区间作为缓冲区, 称为单缓冲区, 以此来缓解两者速度不匹配的矛盾。

当应用层程序操作一个慢速设备时, 操作系统自动为该设备分配一段合适的单缓冲区。对于写操作而言, 应用程序首先操作与设备对应的缓冲区, 通常在缓冲区被写满后,

将缓冲区的数据写入具体设备中。例如串口数据的输出通常以行粒度进行管理，在遇到行分隔符（通常是回车）或者单缓冲区写满时，将内存缓冲区的数据一次性发给串口并清空缓冲区。对于读操作而言，设备驱动可以根据局部性原理将数据预取（Prefetch）入缓冲区中。预取的缓冲区策略可以很好地提升顺序读的速度。操作系统可以根据缓冲区的访问模式，使用类似内存替换策略的方法来管理有限大小的缓冲区，以保证热数据能很好地维持在内存中，降低实际 I/O 的次数。

缓冲区的引入还能解决 I/O 数据处理粒度不匹配的问题。比如，磁盘设备每次读写的单位是固定数量的扇区。频繁从磁盘中读写小数据是低效的，因为实际读写的开销被放大了。为此，在内存中开辟适当大小的缓冲区，避免因磁盘读写引入的延迟，这对于系统性能的提升十分必要。早期 Linux 操作系统中有专门的磁盘缓冲区，现在 Linux 则直接使用文件系统中的页缓存（Page Cache）作为缓冲区，以避免对磁盘设备的频繁访问。

双缓冲区

有时候仅仅设立一个缓冲区并不足以解决设备使用的问题。以显卡的图像绘制为例，显卡有自带的存储空间，俗称显存。为了显示画面，显存需要定期和主存同步，提取待渲染的画面数据。对于视频流播放或电子游戏这类需要高频刷新屏幕的场景而言，应用程序需要不断提供最新画面，而显卡则负责逐帧绘制画面。在显卡和 CPU 共用同一块缓冲区的情况下，如果显存和主存的同步尚未完成，CPU 就将下一帧的画面写入缓冲区中，那么将导致显示的画面数据不一致，乃至出现画面撕裂的现象。此时，就需要引入“双缓冲区”的思路。

双缓冲区指维护两个缓冲区，当第一个缓冲区被填满之后，在它被读取之前，使用第二个缓冲区来收集新的输入。两个缓冲区轮流使用，这一做法被称为缓冲区对换（Buffer Swapping）。如图 13.18 所示，引入双缓冲区可以有效避免 CPU 和显卡在操作画面数据时产生冲突，保证画面渲染的正确性。



图 13.18 CPU 和显卡使用双缓冲区的方法来交换显示数据

双缓冲区还可以用于提高 CPU 和设备交换数据的并行度，将双缓冲区中的一个视为输入缓冲区，另一个视为输出缓冲区，二者的操作互不干扰。此时设备和 CPU 可以互为生产者和消费者。例如在网卡 DMA 缓冲区的配置中，就使用了双缓冲区，将发送队列和接收队列进行了分离。

多缓冲区

当设备的性能足够好，单位时间内能传输大量数据时，两个缓冲区可能无法满足需求，此时就需要分配更多的缓冲区以容纳更多等待交换的数据。多缓冲区有两种常见的组织形式。一种是将缓冲区分散地分配在物理内存的多个位置，每次与设备交互时，将多个缓冲区的基地址和偏移都告诉设备，设备以批处理的方式一次性进行处理。这种方式也被称为向量化 I/O (Vector I/O) 或者分散 - 聚合 I/O (Scatter-Gather I/O)。

另一种组织形式是将多缓冲区组织成环形队列的数据结构，称为环形缓冲区 (Ring Buffer)。环形缓冲区由一段有界的连续内存区域和两个指针组成。一个指针指向有效区域的开始位置，叫作读指针，读取数据时应该从该指针指向的位置开始；另一个指针指向下一个空闲的地址，叫作写指针，也就是新的数据到来时应该被放置的地址。环形缓冲区可以通过指针推进的方法充分利用已有的有界内存空间，避免频繁分配新缓冲区。读取数据时推进读指针，新的数据到来时推进写指针。高速网卡的收发队列和 (基于 NVMe 协议的) SSD 存储设备的命令队列普遍采用环形缓冲区的设计。

小思考

设备缓冲区的使用是必要的吗？

需要注意，合理地利用缓冲区能够有效提升系统的整体性能，但如果数据在被真正处理之前多次进入不同的缓冲区，反而可能因为过多的数据拷贝而降低性能。如何在整个 I/O 软件栈上尽量减少数据拷贝的次数，也是当今操作系统研究者广泛讨论的一个话题。

小知识：假脱机技术

假脱机技术的英文全称是 Simultaneous Peripheral Operation OnLine，简写为 SPOOLing，完整的中文翻译是“外部设备联机并行操作”。假脱机技术结合了缓冲区和队列的思想，针对的是无法被多个用户或进程同时使用的设备，如早先的打印机设备。虽然这项技术现在看来已然过时，但从系统的角度依然有不少值得借鉴的地方。

什么是脱机技术？这里的脱机技术不同于浏览器所谓的“脱机”。浏览器所谓的脱机，指的是在没有联网的情况下，本地缓存了部分网页，使得用户可以在断网的情况下直接浏览本地的网页信息。

而对于 I/O 设备中的脱机，指的是在没有主机参与的情况下完成的输入 / 输出操作，也即“脱离主机”。与脱机 I/O (Offline I/O) 相对的是联机 I/O (Online I/O)。早期的计算机 I/O 过程需要主机的处理器参与，因此属于联机 I/O。但是工程师发现让

高速处理器处理慢速 I/O 并不合理，于是引入了一台“外围控制机”。我们可以将这台“外围控制机”理解为类似 DMA 控制器的装置，旨在将处理器从 I/O 过程中解脱出来。在当时内存还不是很充裕的情况下，计算机工程师将磁盘视为比较高效的缓冲区，用来缓解高速处理器和低速外部设备间的性能鸿沟。借助外围控制机，数据就可以“脱机”写入磁盘，而不消耗主机资源。

什么是“假”脱机技术？假脱机技术没有引入外围控制机这样的设备，而是使用软件模拟的方式，通过操作系统的“假脱机”进程来模拟脱机技术的输入操作与输出操作。该方法本身实际上是一种联机操作，从假脱机的全称 Simultaneous Peripheral Operation OnLine 就可以看出来。

假脱机技术的几点设计思想如下：

- 使用缓冲区：在将数据交给磁带、打印机这类慢速设备之前，先放到磁盘上的缓冲区作为中转。对于当时的慢速设备而言，磁盘被视为类似现在的内存一样的高速缓冲中介（当时的内存资源极为稀缺）。如图 13.19 所示，假脱机技术使用了双缓冲区，分别存放输入文件和输出文件。
- 使用队列：假脱机是操作系统模拟的脱机行为，它为每个用户进程提供逻辑设备接口，比如打印机接口。对于每个进程而言，仍旧是将数据写入逻辑设备接口中。假脱机技术将不同用户进程的请求和数据放入由磁盘模拟的队列中，然后再交给真正的物理设备，保证慢速设备处理数据的正确性。队列的使用使得用户进程不必各自查询设备状态。
- 并行化 I/O：操作系统的多个 I/O 操作可以“同时”发生。例如，从磁带读取数据并进行打印的过程中，操作系统可以在磁带机或打印机任意一方就绪时立刻处理输入文件或输出文件。并行化 I/O 可以有效降低 CPU 等待设备的整体时间，同时提高不同设备之间的数据传输效率。

SPOOLing 并行化 I/O 的目的之一是将数据在两个设备之间做快速搬移，中间需要借助磁盘或内存作为中转站。数据至少经历两次拷贝，有什么方法可以进一步减少这个代价呢？在现代计算机系统中，出现了设备直连的方案，如 NVIDIA GPU 的 GPUDirect 技术^[8]。GPUDirect Storage 允许 GPU 绕过 CPU 直接与 SSD 存储进行交互，而 GPUDirect RDMA 则允许 GPU 直接通过 RDMA 网卡形成 GPU 设备的 P2P 网络，进行高效的数据传输。

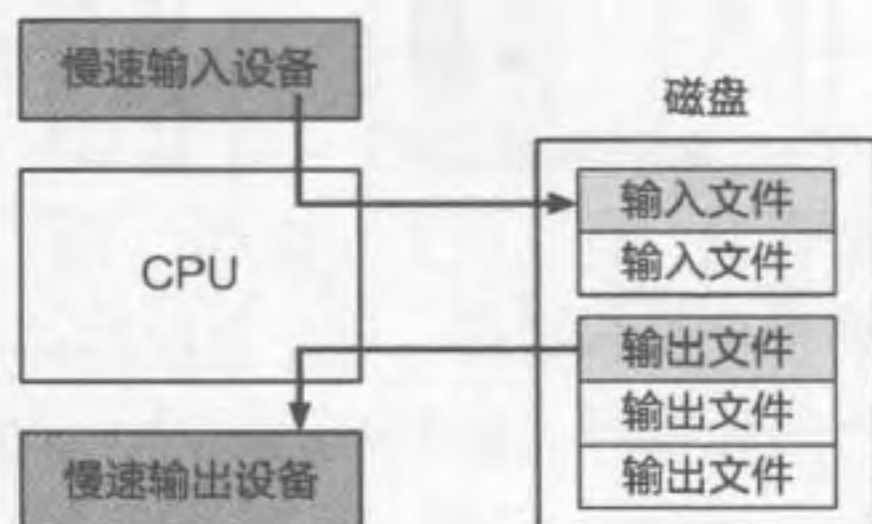


图 13.19 SPOOLing 工作原理示意图
(彩色部分表示当前正在使用的 SPOOL 文件)

Windows 系统上的 Spooler 是什么？出于历史原因，Windows 系统上的打印机守护进程沿用了“Spooler”的名称。严格意义上，“Print Spooler”进程是一个后台服务，该服务使用内存作为缓冲区，所有的打印请求首先加入 Spooler 后台服务的打印队列中，Spooler 后台服务主动检查打印机是否就绪（比如打印机是否被其他机器的请求队列所占用）。如果打印机可用，则主动连接打印机并进行打印服务。Windows 操作系统上的 Print Spooler 组件层次如图 13.20 所示。

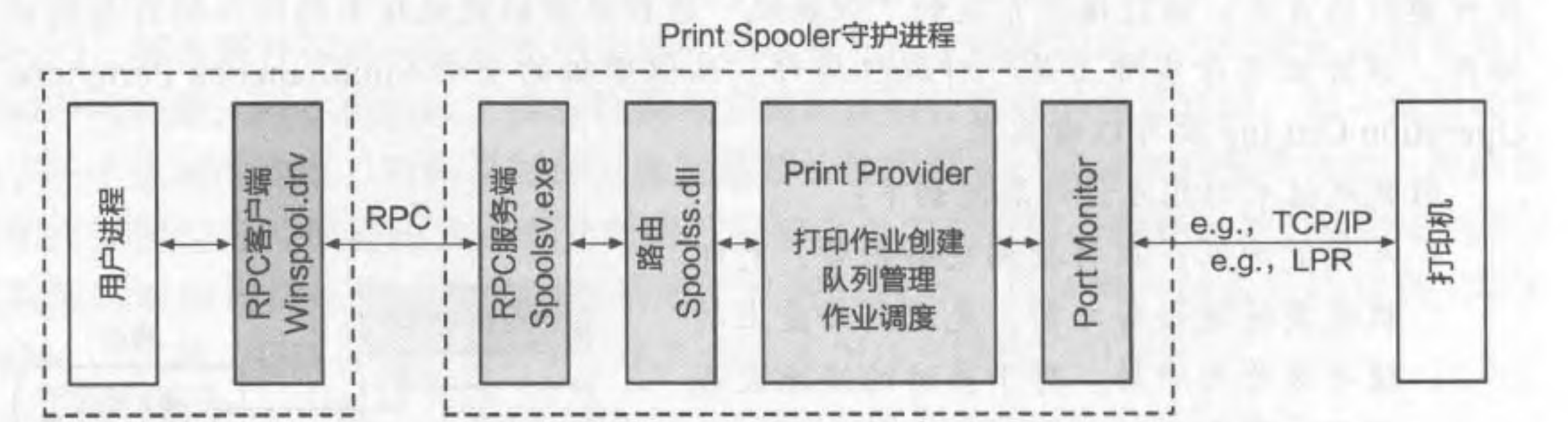


图 13.20 Windows 操作系统上的 Print Spooler 组件层次图^[9]。其中用户进程通过 RPC 和 Print Spooler 守护进程进行通信。守护进程中的 Print Provider 层负责创建打印作业，管理 Spool 文件队列，以及作业调度。Print Spooler 和远端打印机使用特定协议（如 TCP/IP）进行交互

Windows 上的 Print Spooler 保留了 Spool 文件的概念，即任何等待打印的文件首先被放入守护进程的 Spool 文件队列中。守护进程定期检查 Spool 文件队列是否为空，不为空的情况下则创建打印作业（Print Job），并通过特定协议（如 TCP/IP 或 Line Printer Remote, LPR）发送给远端的打印机进行打印。一言以蔽之，Windows 系统的 Spooler 是用户进程和打印机设备之间的代理，负责调度所有用户请求的打印作业，并通过远程过程调用（RPC）和打印机设备进行交互。

13.3.4 设备的使用接口

操作系统除了为设备管理提供共性的功能支持之外，还需要为应用程序提供合适的接口。类 UNIX 操作系统默认将所有设备都抽象为设备文件，并提供文件系统的读写接口（即 read/write）来支持应用程序与设备之间的数据交换。那么，是否所有设备管理功能都能通过 read/write 接口来满足呢？请考虑如下场景：

- 在运行桌面娱乐应用时，电脑游戏需要重新配置显示器的分辨率。
- 在程序开发阶段，调试器需要将串口从中断模式动态调整为轮询模式。
- 在数据中心内部，防火墙软件需要将网卡的工作状态配置为“混杂模式”，以收集更多的网络流量用于分析网络攻击状况。

- 在工业场景中，工控程序在异常情况下需要调整 LED 灯的显示颜色和闪烁频率。

分析以上案例，我们发现 read/write 接口并不足以覆盖设备的所有操作需求，尤其是在控制设备的工作模式方面。为此，类 UNIX 操作系统给设备文件提供了一个足够通用的操作接口，即 I/O Control 接口——ioctl。如代码片段 13.12 所示，ioctl 允许应用程序和设备驱动之间自定义一组操作请求：应用程序在操作设备时，需要同时发送请求码和相应参数，设备驱动则根据请求码的语义调用对应的 I/O 处理逻辑。Windows 操作系统上用于 I/O 控制的接口称为 DeviceIoControl，其作用和 UNIX 系统上的 ioctl 相同。

代码片段 13.12 用于应用程序操作设备的通用接口：ioctl

```
1 #include <sys/ioctl.h>
2
3 int ioctl(int fd, unsigned long request, ...);
```

代码片段 13.13 展示了如何在 UNIX 系统上使用 ioctl 接口周期性地控制 LED 设备的开和关两个动作。应用程序首先将设备文件系统下的 LED 设备文件打开，然后使用 ioctl 对文件描述符进行操作，操作内容为开关动作对应的操作码。操作码是应用程序和设备驱动事先商量好的“魔数”。由于魔数被直接硬编码在应用程序中，该做法在一定程度上损害了应用程序的可移植性。

代码片段 13.13 使用 ioctl 操作 LED 设备

```
1 #define LED_ALL_ON _IO('L', 0x1234)
2 #define LED_ALL_OFF _IO('L', 0x5678)
3
4 int main(void)
5 {
6     int fd = open("/dev/led", O_RDWR);
7     if (fd < 0) exit(1);
8
9     while(1) {
10         ioctl(fd, LED_ALL_ON);
11         sleep(1);
12         ioctl(fd, LED_ALL_OFF);
13         sleep(1);
14     }
15
16     close(fd);
17     return 0;
18 }
```

如代码片段 13.14 所示，LED 驱动程序在收到由应用程序发来的操作码后，会根据不同的操作码内容对 LED 设备进行相应的 MMIO 操作，从而实现 LED 的全亮和全灭。和 read/write 接口一样，设备驱动也需要向操作系统注册 ioctl 接口，这样操作系

统才能正确地调用该接口。

代码片段 13.14 用户程序和设备驱动约定好相同的操作码，设备驱动根据不同请求的操作码执行对应操作

```
1 #define LED_ALL_ON _IO('L', 0x1234)
2 #define LED_ALL_OFF _IO('L', 0x5678)
3
4 long led_ioctl(..., unsigned int cmd, ...)
5 {
6     switch (cmd) {
7         case LED_ALL_ON:
8             *gpio_data |= 0x0; break;
9         case LED_ALL_OFF:
10            *gpio_data &= ~0x0; break;
11         default: break;
12     }
13     ...
14 }
15
16 static struct file_operations fops = {
17     .ioctl = led_ioctl,
18 };
```

13.4 应用 I/O 框架

本节主要知识点

- 为什么要为应用程序提供应用层的 I/O 库？
- 用户态 I/O 管理和操作系统 I/O 管理有什么优点与缺点？

13.4.1 应用层 I/O 库

有时候操作系统没法提供足够适合应用程序特点的 I/O 策略。为了支持定制化的 I/O 策略，操作系统引入了应用层 I/O 库。I/O 库作为应用程序的一部分，通常以共享库的形式直接与应用程序链接，旨在提供更高的性能和更为灵活的设备管理能力。

libc

学过 C 语言的读者应该记得 C 程序使用了 `fopen`、`fclose`、`fread`、`fwrite` 等一系列以 `f` 开头的 I/O 库函数，这组库函数不同于文件系统的系统调用（`open`、`close`、`read`、`write`）。库函数属于高层接口（High-level API），而系统调用接口属于低层接口（Low-level API）。前者返回文件指针，而后者返回内核提供的文件描述符。

请思考代码片段 13.15 和代码片段 13.16 中，哪一段代码的运行效率更高呢？